

XENOGRAM 使い方ガイド

ニコニコ動画の再生数・いいね・マイリスト・コメントを追跡し、Discordへレポートを送るBotです。あわせてボカコレ等のランキング監視と、X（旧Twitter）のキーワード監視を行います。

設定・監視対象・実行時刻はすべてDiscordサーバー（以下「鯖」）ごとに独立しています。 同じBotを複数の鯖に入れて、それぞれ別の運用ができます。

新しい鯖に導入したときは完全に空の状態から始まります。 登録されていない機能は一切動きません（外部APIも叩かず、通知も出しません）。使いたい機能だけを、後述の手順で登録してください。

目次

1. [最初にやること](#)
 2. [コマンド一覧](#)
 3. [自動で動くもの](#)
 4. [一覧からその場で編集する](#)
 5. [動作のカスタマイズ](#)
 6. [鯖ごとに分かれるもの・共有されるもの](#)
 7. [サーバー運用（PM2）](#)
 8. [データベースとマイグレーション](#)
 9. [困ったときは](#)
-

最初にやること

1. 通知先チャンネルを指定する（必須）

```
/guild_setup channel:#通知したいチャンネル
```

これを実行するまで、その鯖には通知が一切届きません。 Botを招待しただけで意図しないチャンネルに投稿が流れ出さないための、意図的な仕様です。

保存前にBotの送信権限（チャンネルを見る／メッセージを送信／埋め込みリンク）を実際に確認します。権限が足りない場合は保存されず、その旨が表示されます。

用途ごとに送信先を分けたい場合は **kind** を指定します。

```
/guild_setup channel:#ranking kind:ボカコレ/ランキング監視  
/guild_setup channel:#x-watch kind:X(Twitter)監視
```

kind を省略すると「通常のお知らせ」（レポート・新着・キリ番・急上昇）の設定になります。ボカコレ用・X用を設定しなかった場合は、通常のお知らせチャンネルにまとめて送られます。

チャンネルを指定せずに実行すると解除になります。

2. 使いたい機能を登録する

登録しなかった機能は動きません。 必要なものだけ設定してください。

投稿者の新着を追いたい場合

```
/user_add user_id:143305795 label:メインアカウント
```

ここに登録したニコニコユーザーの新規投稿を自動検知し、監視リストへ追加します。1人も登録しなければ、新着検知は動作しません（ニコニコのAPIも呼びません）。

特定の動画だけを追いたい場合

```
/add video_id:sm12345678
```

自分以外の投稿者の動画も追えます。投稿者登録と併用できます。

ランキングを監視したい場合

```
/vc_add keyword:曲名 target:曲名
```

キーワードを1件も登録しなければ、ランキング監視は動作しません。

3. 動作確認

```
/status
```

通知先・各機能が「稼働中」か「未設定」かが確認できます。「未設定」と出ている機能は、そのサーバーでは一切動作していません。

何も登録していない状態では

新しい鯖にBotを入れただけの状態では、次のとおり**完全に何も起きません**。

- 定期ジョブは起動するが、対象が無いことを確認して即座に終了する
- ニコニコAPI・ランキングページ・Xの検索はいずれも呼ばれない
- Chromium（スクリーンショット用）も起動しない
- 通知は1件も送られない

他の鯖の設定や監視対象が引き継がれることもありません。

コマンド一覧

`/help` を実行すると、カテゴリ別のメニューから同じ内容を読めます。

コマンドの入力欄（動画ID・キーワードIDなど）は、打ち始めると候補が自動で表示されます。IDをコピーする必要はありません。

動画の監視

コマンド	説明
<code>/stats video_id:<ID></code>	指定動画の現在値・前日比・伸びの傾向・推移グラフを表示
<code>/list</code>	この鯖が監視中の全動画
<code>/add video_id:<ID></code>	動画を監視対象に追加
<code>/remove video_id:<ID></code>	監視リストから外す（統計履歴は残る）
<code>/compare video_id1:<ID> video_id2:<ID></code>	2つの動画を比較（直近24時間の伸びも含む）
<code>/ranking type:<指標></code>	再生・いいね・マイリスト・コメント別のランキング
<code>/growth</code>	直近で最も伸びている動画のトップ5
<code>/upcoming</code>	もうすぐキリ番に届く動画
<code>/export</code>	統計データをCSVで出力

ボカコレ／ランキング監視

登録した曲名・アーティスト名がランキングに載ったら通知します（完全一致）。

コマンド	説明
<code>/vc_add keyword:<語> target:<曲名/アーティスト名></code>	監視キーワードを追加
<code>/vc_list</code>	一覧表示（ ここから各項目を編集できます ）
<code>/vc_remove id:<番号></code>	削除
<code>/vc_check</code>	今すぐ1回実行（管理者）
<code>/vc_toggle state:<on/off></code>	定期監視の有効・無効（管理者）

`/vc_add` の任意オプション:

- `active_from` / `active_until` — 監視する期間（例: `2026-08-21 19:00`）。イベント期間中だけ監視したい場合に使います
- `page_id` — 対象ページの識別子（既定 `rookie`）。複数ランキングを監視する場合に使い分けます
- `note` — メモ

X (Twitter) キーワード監視

読み取り専用です。投稿は一切行いません。 `.env` で `TWITTER_MONITOR_ENABLED=true` にしないとコマンド自体が表示されません。

コマンド	説明
<code>/x_add query:<検索語></code>	監視キーワードを追加
<code>/x_list</code>	一覧表示 (ここから編集できます)
<code>/x_remove id:<番号></code>	削除
<code>/x_check</code>	今すぐ1回実行 (管理者)
<code>/x_toggle state:<on/off></code>	定期監視の有効・無効 (管理者)

監視対象ユーザー

コマンド	説明
<code>/user_add user_id:<数字> label:<メモ></code>	ニコニコユーザーを監視対象に追加 (管理者)
<code>/user_list</code>	一覧表示 (ここから編集できます)
<code>/user_remove user_id:<数字></code>	削除 (管理者)

鯖ごとの設定

コマンド	説明
<code>/guild_setup channel:<#チャンネル> kind:<種別></code>	通知先を設定 (管理者)
<code>/guild_adopt</code>	サーバー未指定で移行された旧データを引き継ぐ (管理者・後述)

動作の調整

コマンド	説明
<code>/set_milestone step:<数></code>	キリ番の判定単位（管理者）
<code>/set_spike threshold:<数> cooldown_hours:<時間></code>	急上昇のしきい値（管理者）
<code>/set_rank_threshold positions:<順位></code>	ボカコレ順位変動の通知しきい値（管理者）
<code>/set_schedule job:<ジョブ> cron:<時刻></code>	定期実行の時刻（管理者）
<code>/settings</code>	現在の設定値を一覧表示

管理・確認

コマンド	説明
<code>/status</code>	稼働状況・通知先・各ジョブの最終実行
<code>/ping</code>	応答速度
<code>/force_update</code>	毎時の処理を今すぐ実行（管理者）
<code>/daily_report</code>	デイリーレポートを今すぐ実行（管理者）
<code>/logs lines:<行数> type:<種別></code>	直近のログを表示（管理者）
<code>/restart</code>	Botを再起動（管理者・PM2運用時のみ自動復帰）
<code>/help</code>	コマンド一覧

【管理者】と書かれたコマンドは、Discordの管理者権限を持つ人にしか表示・実行できません。またこのBotはDMでは使えません（設定が鯖単位のため）。

自動で動くもの

すべて日本時間（Asia/Tokyo）で動作し、**実行時刻は鯖ごとに変更できます。**

ジョブ	既定の時刻	内容
新着／キリ番／急上昇チェック	毎時0分	新規投稿の検知、キリ番突破の通知、急上昇の検知、統計の記録
デイリーレポート	毎朝7時	動画ごとの前日比・伸びの傾向・グラフ、全体サマリー
ボカコレ／ランキング監視	毎時5分	登録キーワードがランキングに載ったかを照合
週次まとめレポート	毎週日曜21時	7日間の伸びの総括
X キーワード監視	毎時10分	登録クエリで検索し、新規ヒットを通知

デイリーレポートの中身

先頭に全体サマリー（合計増加・伸びTOP5・勢いが上下した動画）が届き、続いて動画ごとに次の内容が送られます。

- 現在値・増減・増加率の表
- 勢い（直近7日の合計／1日平均／直近1日 → 加速中・横ばい・減速中）
- 次のキリ番までの残数と到達予測日
- 1日の増加量（棒）と累計（線）を重ねた推移グラフ

「勢い」は増加量が2日ぶん揃ってから表示されます（監視開始から3日目以降）。比較できる過去の記録が無い間は、増減欄が -- になります。

ボカコレ監視ON時の連動

`/vc_toggle state:on` にすると、その鯖のデイリーレポートが自動的に「毎時20分」へ切り替わります。イベント期間中に伸びを細かく追うためです。 `off` に戻すと、ONにする直前の設定に自動復元されます。

一覧からその場で編集する

`/vc_list` `/x_list` `/user_list` には、一覧の下に編集用のメニューが付きます。

- 1. メニューから項目を選ぶ
- 2. 現在の値が入った入力フォームが開く
- 3. 書き換えて保存すると、一覧がその場で更新され、変更点が自分だけに表示される

編集できる項目:

一覧	編集できる項目
<code>/vc_list</code>	キーワード／対象（曲名・アーティスト）／ページID／監視期間／監視のon-off
<code>/x_list</code>	検索クエリ／メモ／監視のon-off
<code>/user_list</code>	ユーザーID／メモ

監視期間は `2026-08-21 19:00 ~ 2026-08-24 17:00` の形式で入力します。空欄にすると「常時」になります。開始だけ、終了だけの指定も可能です。

メニューを操作できるのは、そのコマンドを実行した本人だけです。

動作のカスタマイズ

すべて**鯖ごと**に保存されます。

ここで扱うのは「キリ番の単位」「急上昇のしきい値」「実行時刻」といった**調整値**です。変更していない項目は `.env` の既定値が使われます（調整値なので、共通の初期値から始まって問題ありません）。一方で**監視対象そのもの**（投稿者・動画・キーワード）は `.env` から引き継がれず、鯖ごとに登録しない限り何も監視しません。

実行時刻を変える

```
/set_schedule job:デیلیーレポート cron:9時30分
```


cron は自然な日本語で入力できます。

ジョブの種類	入力例
毎時のジョブ	5 5分（毎時5分）
毎日のジョブ	7時 7時30分 7:30
毎週のジョブ	日 21時 月曜9時30分

cron式（30 9 * * * など）をそのまま入力することもできます。変更は再起動なしで即座に反映されます。

キリ番の単位

```
/set_milestone step:1000
```

再生数が増えてきたら 100 → 1000 のように上げてください。

急上昇の判定

```
/set_spike threshold:500 cooldown_hours:6
```

threshold は「1時間あたりの再生増加数」です。実行間隔に関わらず1時間換算で判定するため、**/set_schedule** で実行間隔を変えても判定基準は変わりません。**cooldown_hours** は同じ動画へ再通知するまでの間隔です。

現在の設定を見る

```
/settings
```

「(デフォルト)」と付いている項目は、その鯖でまだ変更していない項目です。

鯖ごとに分かれるもの・共有されるもの

対象	扱い
監視動画リスト	鯖ごと
ボカコレ監視キーワード	鯖ごと
X 監視キーワード	鯖ごと
監視ニコニコユーザー	鯖ごと
各種設定値・実行時刻	鯖ごと
通知先チャンネル	鯖ごと
再生数などの統計履歴	動画単位で全鯖共有

統計を共有にしている理由は、再生数がニコニコ側の事実であって鯖によって変わらないためです。これにより次の利点があります。

- 同じ動画を複数の鯖が監視しても、API取得もDB容量も1本ぶんで済む
- 後から監視を始めた鯖も、その動画の過去の推移グラフをすぐに見られる

一方で「どこまで通知したか」は鯖ごとに独立して記録しているため、先に処理が走った鯖の影響でギリ番通知を取りこぼすことはありません。

同じキーワードや同じ動画を別々の鯖で登録することもできます。他の鯖の登録内容を読んだり書き換えたりすることはできません。

サーバー運用（PM2）

必要環境

- Node.js **22.12.0 以上**（組み込みの `node:sqlite` を使うため）
- Discord Bot トークン
- ボカコレ監視のスクリーンショット機能を使う場合は Chromium（Puppeteer 同梱版で可）

初回セットアップ

```
npm install
cp .env.example .env # 内容を埋める
```

```
npm run pm2:start
```

日常操作

コマンド	内容
<code>npm run pm2:start</code>	起動
<code>npm run pm2:restart</code>	再起動（コード変更後は必須）
<code>npm run pm2:stop</code>	停止
<code>npm run pm2:logs</code>	ログを追う
<code>npm run pm2:status</code>	稼働状態

PM2 の設定（`ecosystem.config.js`）では次の面倒をしています。

- クラッシュ時の自動再起動（10回まで、再試行のたびに待ち時間を延長）
- メモリが2GBを超えたら再起動
- 毎日午前4時の予防的な再起動（他のジョブと重ならない時刻）
- 停止時は15秒の猶予を取り、撮影中のChromiumを閉じてから終了

主な環境変数

`.env.example` に全項目があります。最低限必要なのは次の4つです。

変数	内容
<code>DISCORD_TOKEN</code>	Botのトークン
<code>DISCORD_CLIENT_ID</code>	アプリケーションID
<code>DISCORD_GUILD_ID</code>	コマンドを登録する鯖（未設定ならグローバル登録）
<code>DISCORD_CHANNEL_ID</code>	マルチテナント化前のデータを移行するときの初期通知先。新規導入時は不要（ <code>/guild_setup</code> で指定）

その他は既定値で動きます。用途に応じて調整してください。

- `NICO_USER_IDS` — **廃止**。監視する投稿者は `/user_add` で鯖ごとに登録します（実行時に参照されません）
- `MILESTONE_STEP` — キリ番の単位の初期値
- `VOCACOLLE_RANKING_URLS` — 監視するランキングページのURL（カンマ区切りで複数可）
- `TWITTER_MONITOR_ENABLED` — X監視を使う場合のみ `true`

- **PORT** — 死活監視用HTTPエンドポイントのポート（既定 47810）

データベースとマイグレーション

データは **data/xenogram.sqlite** に保存されます（Node.js 組み込みの SQLite。追加の依存なし）。

スキーマ変更のしかた

構造を変える場合は **migrations/** 配下にSQLファイルを追加してください。ファイル名の昇順に、未適用のものだけが起動時に自動適用され、適用済みのバージョンは **schema_migrations** テーブルに記録されます。

```
migrations/0004_なにをするか.sql
```

services/repositories/db.js にある **CREATE TABLE IF NOT EXISTS** 群は「初期スキーマ」で、**起動のたびに実行されます**。ここを直接書き換えたり、マイグレーションで削除したテーブルを残しておいたりすると、次回起動時に空のテーブルが復活します。構造変更は必ず **migrations/** 側で行ってください。

適用は「外部キー検査を止める → トランザクション開始 → 実行 → 記録 → コミット」の順で行われ、失敗した場合はロールバックされます。適用後に外部キー違反の有無も確認されます。

検証用のDB切り替え

本番DBに触れずに動作を試したい場合は、環境変数でDBファイルを差し替えられます。

```
XENOGRAM_DB_PATH=/path/to/test.sqlite node -e "require('./services/repositories/db')"
```

バックアップ

WALモードのため、ファイルを単純にコピーすると最新データが欠ける可能性があります。一貫したスナップショットを取るには **VACUUM INTO** を使ってください。

```
node -e "require('./services/repositories/db').db.exec(\"VACUUM INTO 'backup.sqlite'\")"
```

困ったときは

通知が届かない

`/status` で「このサーバーの通知先」を確認してください。「未設定」と出ている場合は `/guild_setup channel:#チャンネル` を実行します。

設定済みなのに届かない場合は、Botがそのチャンネルへ投稿する権限（チャンネルを見る／メッセージを送信／埋め込みリンク）を持っているか確認してください。

コマンドが表示されない

- 【管理者】のコマンドは管理者権限を持つ人にしか表示されません
- X関連のコマンドは `.env` で `TWITTER_MONITOR_ENABLED=true` にしないと表示されません
- コマンドの追加・説明変更は、Bot再起動時に自動で再登録されます

DMで使えない

設定・監視対象が鯖単位で管理されているため、DMでは実行できません。サーバー内で使ってください。

グラフが表示されない

グラフは2日ぶん以上の記録が必要です。監視を始めた直後は表示されません。

「前日比」が -- になる

比較できる過去の記録がまだ無い状態です。増減が無い (± 0) のとは区別して表示しています。次回のレポートから差分が出ます。

旧データが見当たらない

`DISCORD_GUILD_ID` を設定せずに起動した場合、マルチテナント化以前のデータは どの鯖にも属さない `legacy` として退避されます。引き継ぎたい鯖で次を実行してください。

```
/guild_adopt
```

定期実行が動いていない

`/status` で各ジョブの最終実行時刻を確認してください。「(再起動後まだ未実行)」の場合は、まだその時刻が来ていないだけの可能性があります。`/settings` で実行時刻を確認し、必要なら `/set_schedule` で変更してください。

手動実行がスキップされる

「前回の処理がまだ終わっていません」と表示された場合、同じ処理が実行中です。二重通知や統計の二重記録を防ぐための制御なので、少し待ってから再実行してください。